Implementation and Verification of the Full UART System

1 Objective

The folder `VERIFICATION/uart-full/` extends the previous transmitter-only work into a complete UART subsystem. The final result is a small but complete 8N1 UART solution composed of:

- a transmitter in `src/uart_tx.v`,
- a receiver in `src/uart_rx.v`,
- and a top-level wrapper in `src/top.v`.

The goal of this report is to explain both the implementation and the verification strategy of that full system. A special point in this assignment is the receive path: the external UART input `rx` does not arrive aligned to the local system clock, so the receiver must handle a clock-domain crossing issue before it can decode serial data safely.

2 Final Files

The final `uart-full` folder contains:

```
src/uart_tx.v
src/uart_rx.v
src/top.v
tb/tb_uart_tx.v
tb/tb_uart_rx.v
tb/tb_top.v
```

The three source files implement the system, and the three testbenches verify it at different levels:

- `tb_uart_tx.v`: dedicated verification of the transmitter,
- `tb_uart_rx.v`: dedicated verification of the receiver,
- `tb_top.v`: integration verification of the full UART system.

3 UART System Overview

The implemented design is a basic UART with the classic 8N1 frame format:

- 1 start bit with value `0`,
- 8 data bits,
- 1 stop bit with value `1`.

The transmitter sends data LSB first, and the receiver reconstructs data in that same order.

Top-Level Interface

```
module top #(
    parameter CLK_FREQ = 27_000_000,
    parameter BAUD_RATE = 115200
)(
    input wire clk,
    input wire rst_n,
    input wire tx_en,
    input wire [7:0] tx_data,
    input wire rx,
    output wire tx,
    output wire tx_busy,
    output wire [7:0] rx_data,
    output wire rx_done,
    output wire rx_busy,
    output wire rx_frame_error
);
```

Two important architectural decisions are visible here:

- `rst_n` is an active-low reset shared by the TX and RX blocks.
- `tx` and `rx` are independent serial lines, so the system is full duplex.

That means the design can transmit and receive at the same time.

4 Timing Parameters

The TX and RX modules both use the same timing definition:

```
localparam integer CLOCKS_PER_BIT = CLK_FREQ / BAUD_RATE;
```

With the default values:

- `CLK_FREQ = 27_000_000`
- `BAUD_RATE = 115200`

the design obtains:

$$\texttt{CLOCKS\_PER\_BIT} = \left\lfloor \frac{27\,000\,000}{115200} \right\rfloor = 234$$

As in the transmitter-only version, this is integer division, so the generated bit period is an approximation of the requested baud rate.

In the full version, both TX and RX also compute an internal counter width from `CLOCKS_PER_BIT`. This avoids a fixed-size counter and makes the design more robust if the parameters are changed later.

5 Architecture of the Full UART System

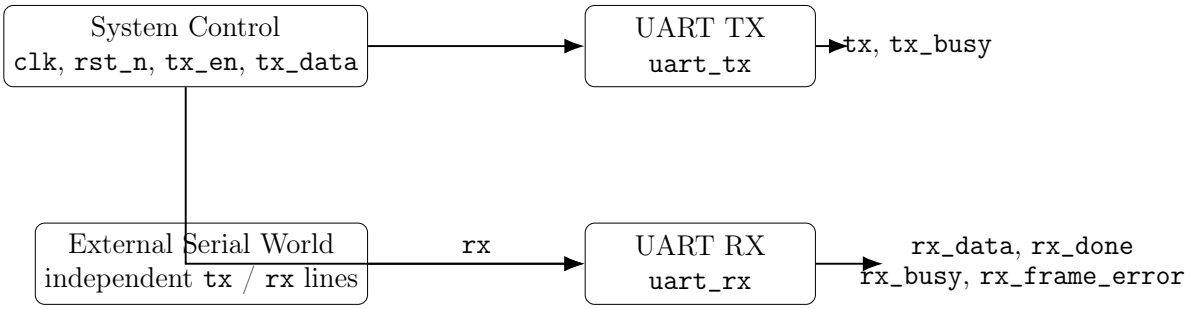

Figure 1: High-level structure of the full UART system.

The top-level module itself is intentionally simple. It does not add another state machine. Instead, it instantiates one transmitter and one receiver in parallel, both driven by the same `clk` and the same active-low reset.

Because the data directions are independent, the design is naturally full duplex:

- the TX block can shift out a frame on `tx`,
- while the RX block simultaneously samples a different frame on `rx`.

6 Transmitter Role in the Full System

The transmitter keeps the same 8N1 behavior already verified in the previous UART work:

- it waits in `s_IDLE`,
- it captures `tx_data` when `tx_en` is accepted,
- it transmits start, 8 data bits, and stop,
- and it keeps `tx_busy` asserted during the active frame.

In the `uart-full` version, the main implementation improvement is the counter width calculation. Instead of assuming a fixed 16-bit counter, the module computes a width from `CLOCKS_PER_BIT`. That makes the timing logic cleaner and avoids width-mismatch issues when linting or changing parameters.

7 Receive Clock-Domain Issue

The receive path is the most delicate part of the full system.

Why There Is a Clock-Domain Problem

The external `rx` signal comes from another device or another timing source. Even if both systems use the same nominal baud rate, the edge on `rx` does not arrive synchronized to the local `clk`. Therefore:

- the line can change very close to a clock edge,
- a flip-flop that samples it can enter metastability,
- and direct single-stage sampling can produce unreliable bit detection.

This is a classic clock-domain crossing issue: the UART line is effectively asynchronous with respect to the receiver clock.

How the Design Solves It

The implemented receiver uses a two-stage synchronizer:

```
reg rx_meta;
reg rx_sync;
reg rx_sync_prev;

rx_meta <= rx;
rx_sync <= rx_meta;
rx_sync_prev <= rx_sync;
```

This solves two practical problems:

- `rx_meta` and `rx_sync` reduce the metastability risk by resynchronizing the input to the local clock domain,
- `rx_sync_prev` makes it possible to detect the synchronized falling edge of the start bit.

The start condition is recognized with:

```
if (rx_sync_prev && !rx_sync)
```

That edge only causes a transition into the receive sequence after the signal has already been brought into the local clock domain.

Why Mid-Bit Sampling Is Used

After detecting a possible start edge, the receiver does not immediately trust the frame. It first waits approximately half a bit time:

```
localparam integer START_TICKS = (CLOCKS_PER_BIT > 1) ?
                                 (CLOCKS_PER_BIT / 2) : 1;
```

Then it checks whether the line is still low. This has two benefits:

- it confirms that the start bit is real and not just a short glitch,
- it places the sampling point near the middle of the start bit.

Once the start bit is confirmed, the receiver samples each data bit once per bit period. In this implementation, that means one sample every `CLOCKS_PER_BIT` clock cycles. This places the sampling point near the center of each bit period and makes the UART receiver tolerant to small phase differences between the transmitting and receiving clocks.

8 Receiver State Machine

The receive state machine has four states:

- `s_IDLE`
- `s_START`
- `s_DATA`
- `s_STOP`

s_IDLE

In `s_IDLE`, the receiver waits for the synchronized falling edge of the start bit. During this state:

- `rx_busy = 0`,
- the counters are cleared,
- and the line is continuously observed for a new frame.

s_START

In `s_START`, the receiver waits half a bit time and checks that the line is still low. If it is no longer low, the event is treated as a false start and the FSM returns to `s_IDLE`.

s_DATA

In `s_DATA`, the receiver waits one full bit time between samples and stores the synchronized line value into:

```
rx_shift[bit_index] <= rx_sync;
```

Because `bit_index` starts at 0, the receiver reconstructs the byte in LSB-first order, matching the transmitter.

`s_STOP`

In `s_STOP`, the receiver samples the stop bit after one bit time:

- if the line is high, the frame is accepted, `rx_data` is updated, and `rx_done` pulses for one clock,
- if the line is low, a framing error is reported with a one-cycle `rx_frame_error` pulse.

9 Top Module Behavior

The file `src/top.v` simply connects the two previously described blocks:

- `uart_tx` handles outgoing traffic,
- `uart_rx` handles incoming traffic,
- both share `clk` and `rst_n`,
- and both operate independently.

This independence is what makes the module full duplex. The top-level block does not force loopback internally. Instead, it exposes one TX line and one RX line. Loopback is only created inside the integration testbench when that specific verification scenario is desired.

10 Why the Testbenches Use Reduced Parameters

The implementation is parameterized for a realistic default clock and baud rate, but the testbenches use smaller values:

```
localparam integer CLK_FREQ = 100_000;
localparam integer BAUD_RATE = 1_000;
localparam integer CLOCKS_PER_BIT = CLK_FREQ / BAUD_RATE;
```

Therefore:

$$\texttt{CLOCKS\_PER\_BIT} = 100$$

This keeps the verification cycle-accurate while reducing simulation time. The clock is still two orders of magnitude faster than the baud rate, so the timing relationship remains representative.

11 Verification Strategy

The full solution is verified at three levels.

1. Dedicated TX Verification

`tb/tb_uart_tx.v` keeps the self-checking strategy from the previous work. It verifies:

- reset behavior,
- nominal frame timing,
- LSB-first ordering,
- data capture semantics,
- busy overlap behavior,
- held-high reacceptance semantics,
- and back-to-back transfers.

Its checking style is cycle-based: it verifies entire bit windows and keeps `tx_busy` under observation throughout the frame.

2. Dedicated RX Verification

`tb/tb_uart_rx.v` focuses on the receive path and, importantly, drives the serial line asynchronously with respect to the local clock. That is why the stimulus tasks use small start offsets such as 1, 2, 3, 4, 5, 6, and 7 ns rather than always changing `rx` exactly on a clock edge.

This is important because it exercises the exact problem the receiver is supposed to solve: a serial input that is not phase-aligned to `clk`.

The RX testbench verifies:

- reset behavior,
- a nominal asynchronously timed frame,
- multiple data patterns,
- rejection of a false start,
- framing error detection,
- and back-to-back legal receptions.

3. Top-Level Integration Verification

`tb/tb_top.v` verifies the combined system and proves that the wrapper is really full duplex.

It includes three different integration scenarios:

- an external receive path using `ext_rx`,
- a simultaneous independent TX and RX case,
- and a loopback mode used only inside the testbench.

The full-duplex case is the most important integration check. In that test:

- the transmitter sends one byte,
- the receiver samples another byte on an independent RX line,
- both operations overlap in time,
- and the testbench confirms that TX and RX can remain active simultaneously.

12 Summary of Tested Cases

TX Testbench Cases

Case	Purpose
1	Reset behavior
2	Single nominal transmission
3	Pattern sensitivity and LSB-first ordering
4	Data capture vs later `tx_data` changes
5	Short overlapping request while busy is ignored
6	Held-high `tx_en` causes legal immediate reacceptance
7	Back-to-back legal transmissions

RX Testbench Cases

Case	Purpose
1	Reset behavior
2	Single asynchronous nominal reception
3	Pattern sensitivity and LSB-first decoding
4	False start is rejected
5	Bad stop bit raises framing error
6	Back-to-back legal receptions

Top Testbench Cases

Case	Purpose
1	Active-low reset behavior
2	External receive path through top
3	Simultaneous independent TX and RX
4	Single TX to RX loopback
5	Back-to-back loopback transfers

13 Validation Command

The final validation was executed from the `VERIFICATION/` directory with:

```
./builder-verification.sh -r uart-full -x -w
```

That flow:

- compiled all three testbenches,
- executed all simulations,
- generated the waveform files,
- and launched GTKWave in the background.

The generated waveforms were:

- `uart-full/build/tb_top.vcd`
- `uart-full/build/tb_uart_rx.vcd`
- `uart-full/build/tb_uart_tx.vcd`

14 Final Execution Results

The final simulation log reported:

Testbench	Cases	Checks	Failures
`tb_top`	5	100	0
`tb_uart_rx`	6	121	0
`tb_uart_tx`	7	24106	0

In total, the complete verification campaign executed:

- 18 highlighted cases,
- 24327 individual checks,
- and 0 failures.

This means:

- the transmitter behavior is correct,
- the receiver correctly handles asynchronous sampling, false starts, and stop-bit errors,
- and the top-level wrapper correctly supports active-low reset and full-duplex operation.

15 Conclusion

The final `uart-full` implementation is no longer only a UART transmitter; it is a complete UART subsystem with:

- a parameterized TX path,
- a synchronized RX path,
- and a simple full-duplex top-level integration.

The most important design addition is the receiver clock-domain handling. The external `rx` line is asynchronous to the local clock, so the design first synchronizes it and then samples near the centers of the serial bits. That makes the receiver much more realistic and robust than a direct raw-input sampler.

The verification strategy matches that design intent. Instead of using only a single integration test, the work verifies:

- the transmitter in isolation,
- the receiver in isolation,
- and the composed top-level system.

Because all three self-checking testbenches passed with zero failures, the final result gives confidence not only in nominal UART transfers, but also in reset behavior, asynchronous receive timing, framing-error reporting, back-to-back operation, and true simultaneous transmit/receive behavior.